Autómatas Finitos

Introducción a la Lógica y la Computación (3era Parte)

Docentes: Mariana Badano, Facundo Bustos, Gervasio Figueroa, Joel Kuperman, Miguel Pagano, Mauricio Tellechea.

Año 2025

Autómata Finito (AF)

- Es un modelo computacional basado en la noción de **máquina secuencial automática** y cuyo poder computacional caracteriza a los lenguajes regulares.

Autómata Finito (AF)

- Es un modelo computacional basado en la noción de **máquina secuencial automática** y cuyo poder computacional caracteriza a los lenguajes regulares.
- Primero, estudiaremos los autómatas finitos en si mismos, es decir, su funcionamiento, sus tipos y sus propiedades.

Autómata Finito (AF)

- Es un modelo computacional basado en la noción de **máquina secuencial automática** y cuyo poder computacional caracteriza a los lenguajes regulares.
- Primero, estudiaremos los autómatas finitos en si mismos, es decir, su funcionamiento, sus tipos y sus propiedades.
- Para luego, probar que efectivmante es la máquina secuencial que caracteriza a los lenguajes regulares.

Qué computa un AF?

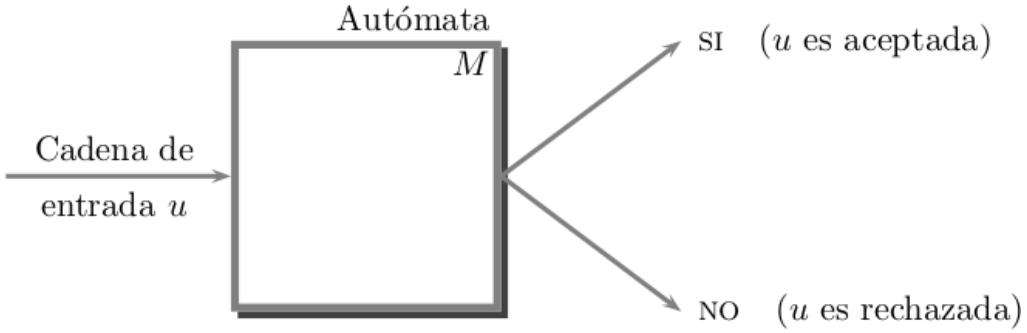
- Un AF computa un lenguaje, en el sentido que decide pertenencia de cadenas a un cierto lenguaje.

Qué computa un AF?

- Un AF computa un lenguaje, en el sentido que decide pertenencia de cadenas a un cierto lenguaje.
- Más precisamente, toma como input una cadena y luego de procesar todos sus símbolos decide si la cadena es **“aceptada”** o **“rechazada”**.

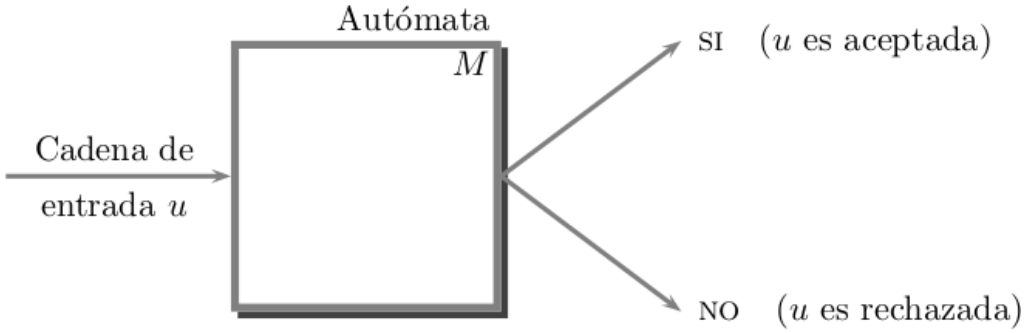
Qué computa un AF?

- Un AF computa un lenguaje, en el sentido que decide pertenencia de cadenas a un cierto lenguaje.
- Más precisamente, toma como input una cadena y luego de procesar todos sus símbolos decide si la cadena es **"aceptada"** o **"rechazada"**.



Qué computa un AF?

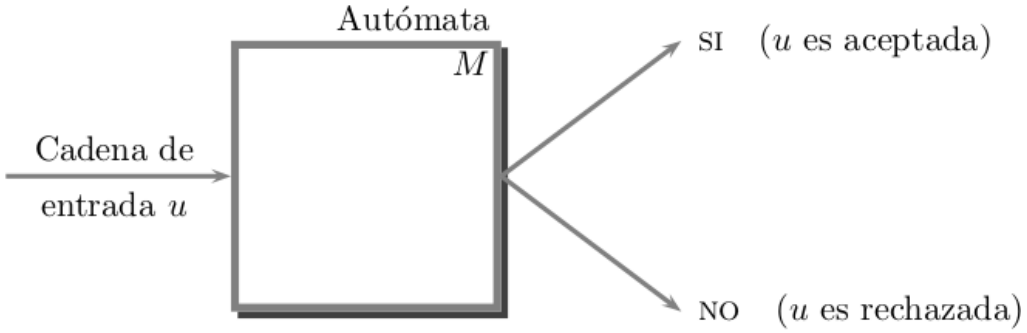
- Un AF computa un lenguaje, en el sentido que decide pertenencia de cadenas a un cierto lenguaje.
- Más precisamente, toma como input una cadena y luego de procesar todos sus símbolos decide si la cadena es **"aceptada"** o **"rechazada"**.



- Estudiaremos 3 tipos distinto de autómatas finitos:
  - Determinísticos (**AFD**)
  - No-Determinísticos (**AFN**)
  - No-Determinísticos con transiciones espontáneas (**AFN**ϵ)

Qué computa un AF?

- Un AF computa un lenguaje, en el sentido que decide pertenencia de cadenas a un cierto lenguaje.
- Más precisamente, toma como input una cadena y luego de procesar todos sus símbolos decide si la cadena es **"aceptada"** o **"rechazada"**.



- Estudiaremos 3 tipos distinto de autómatas finitos:
  - Determinísticos (**AFD**)
  - No-Determinísticos (**AFN**)
  - No-Determinísticos con transiciones espontáneas (**AFN**ϵ)
- Aunque todos **computacionalmente equivalentes** entre si.

Cómo funciona un AF?

- Un AF cuenta con un conjunto finito de **"estados"** y en todo momento la máquina se encuentra en exactamente uno de ellos.

Cómo funciona un AF?

- Un AF cuenta con un conjunto finito de **“estados”** y en todo momento la máquina se encuentra en exactamente uno de ellos.
  - Hay un único **“estado inical”** desde donde comienza todo computo de la máquina.

Cómo funciona un AF?

- Un AF cuenta con un conjunto finito de **"estados"** y en todo momento la máquina se encuentra en exactamente uno de ellos.
  - Hay un único **"estado inical"** desde donde comienza todo computo de la máquina.
  - Hay un subconjunto de estados llamados **"estados de aceptacion o finales"** que definen si una cadena es aceptada por la máquina.

Cómo funciona un AF?

- Un AF cuenta con un conjunto finito de **"estados"** y en todo momento la máquina se encuentra en exactamente uno de ellos.
  - Hay un único **"estado inical"** desde donde comienza todo computo de la máquina.
  - Hay un subconjunto de estados llamados **"estados de aceptacion o finales"** que definen si una cadena es aceptada por la máquina.
- Además, cuenta con una cinta, llamada **"cinta de entrada"**, donde en cada posición o celda de la cinta se alojará un símbolo de la cadena a procesar, y un cabezal de solo lectura que apunta a dichas celdas.

Cómo funciona un AF?

- Un AF cuenta con un conjunto finito de **"estados"** y en todo momento la máquina se encuentra en exactamente uno de ellos.
  - Hay un único **"estado inical"** desde donde comienza todo computo de la máquina.
  - Hay un subconjunto de estados llamados **"estados de aceptacion o finales"** que definen si una cadena es aceptada por la máquina.
- Además, cuenta con una cinta, llamada **"cinta de entrada"**, donde en cada posición o celda de la cinta se alojará un símbolo de la cadena a procesar, y un cabezal de solo lectura que apunta a dichas celdas.
- Cada vez que el cabezal lee un símbolo de la cinta de entrada, la máquina ejecuta un cambio de estado, llamada **"transición"**, y el cabezal se desplaza hacia la celda contigua a procesar. Así sucesivamente, hasta que el cabezal lee todos los símbolos de la cadena que ocurre en la cinta de entrada.

Cómo funciona un AF?

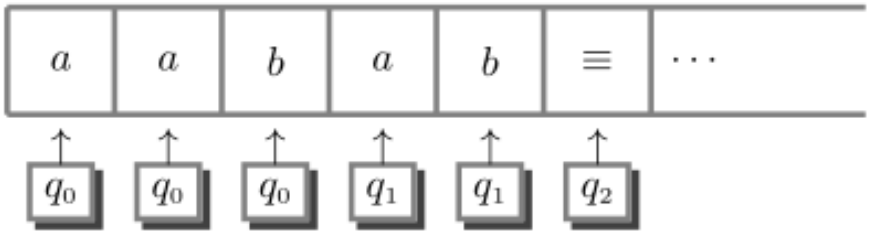
- Un AF cuenta con un conjunto finito de **"estados"** y en todo momento la máquina se encuentra en exactamente uno de ellos.
  - Hay un único **"estado inical"** desde donde comienza todo computo de la máquina.
  - Hay un subconjunto de estados llamados **"estados de aceptacion o finales"** que definen si una cadena es aceptada por la máquina.
- Además, cuenta con una cinta, llamada **"cinta de entrada"**, donde en cada posición o celda de la cinta se alojará un símbolo de la cadena a procesar, y un cabezal de solo lectura que apunta a dichas celdas.
- Cada vez que el cabezal lee un símbolo de la cinta de entrada, la máquina ejecuta un cambio de estado, llamada **"transición"**, y el cabezal se desplaza hacia la celda contigua a procesar. Así sucesivamente, hasta que el cabezal lee todos los símbolos de la cadena que ocurre en la cinta de entrada.
- El estado al que arriba la máquina, ya sea, de aceptación o no, luego de procesar todos los simbolos de la cadena, definirá si la cadena es **aceptada o rechazada**, respectivamente.

Cómo funciona un AF?

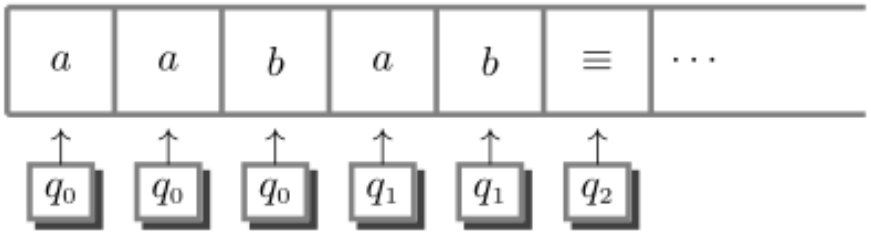
- Un AF cuenta con un conjunto finito de **"estados"** y en todo momento la máquina se encuentra en exactamente uno de ellos.
  - Hay un único **"estado inical"** desde donde comienza todo computo de la máquina.
  - Hay un subconjunto de estados llamados **"estados de aceptacion o finales"** que definen si una cadena es aceptada por la máquina.
- Además, cuenta con una cinta, llamada **"cinta de entrada"**, donde en cada posición o celda de la cinta se alojará un símbolo de la cadena a procesar, y un cabezal de solo lectura que apunta a dichas celdas.
- Cada vez que el cabezal lee un símbolo de la cinta de entrada, la máquina ejecuta un cambio de estado, llamada **"transición"**, y el cabezal se desplaza hacia la celda contigua a procesar. Así sucesivamente, hasta que el cabezal lee todos los símbolos de la cadena que ocurre en la cinta de entrada.
- El estado al que arriba la máquina, ya sea, de aceptación o no, luego de procesar todos los simbolos de la cadena, definirá si la cadena es **aceptada o rechazada**, respectivamente.

Procesando la cadena “aabab” en un AF

Procesando la cadena “aabab” en un AF

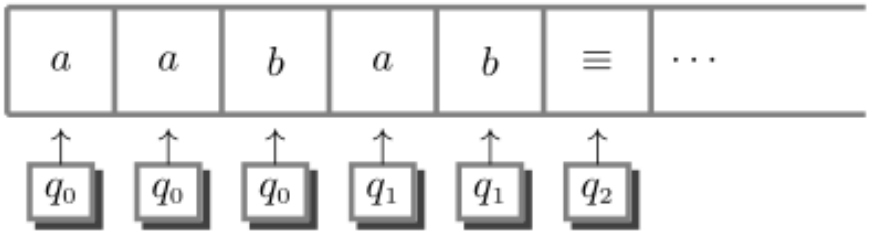


Procesando la cadena “aabab” en un AF



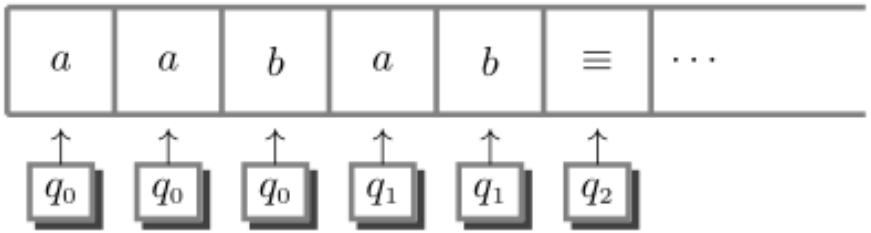
1. El computo arranca siempre en el estado inicial q_0 de la máquina.

Procesando la cadena “aabab” en un AF



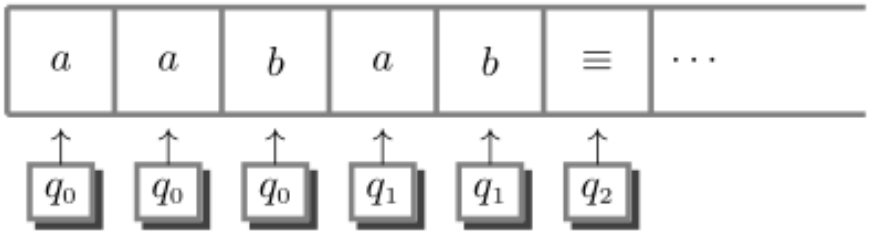
1. El computo arranca siempre en el estado inicial q_0 de la máquina.
2. El cabezal lee el primer simbolo ‘a’ y la máquina transiciona al estado q_0.

Procesando la cadena “aabab” en un AF



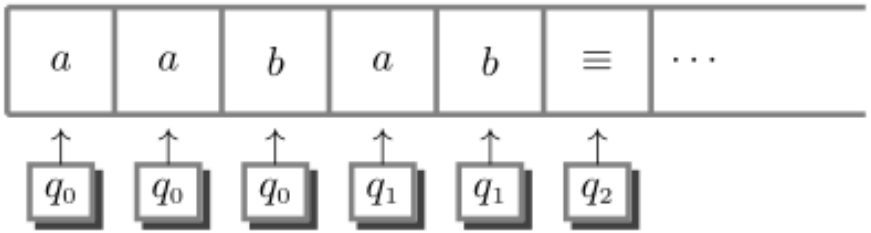
1. El computo arranca siempre en el estado inicial q_0 de la máquina.
2. El cabezal lee el primer simbolo ‘a’ y la máquina transiciona al estado q_0.
3. El cabezal lee el segundo simbolo ‘a’ y la máquina transiciona al estado q_0.

Procesando la cadena “aabab” en un AF



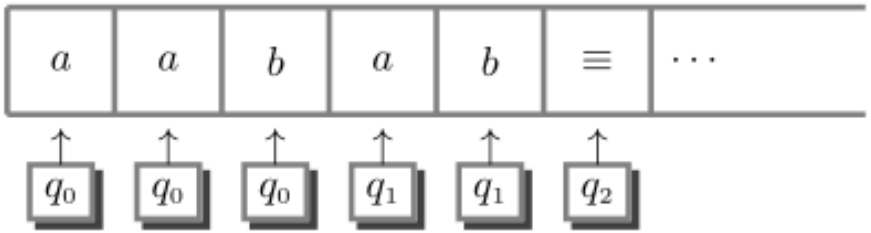
1. El computo arranca siempre en el estado inicial q_0 de la máquina.
2. El cabezal lee el primer simbolo ‘a’ y la máquina transiciona al estado q_0.
3. El cabezal lee el segundo simbolo ‘a’ y la máquina transiciona al estado q_0.
4. El cabezal lee el tercer simbolo ‘b’ y la máquina transiciona al estado q_1.

Procesando la cadena “aabab” en un AF



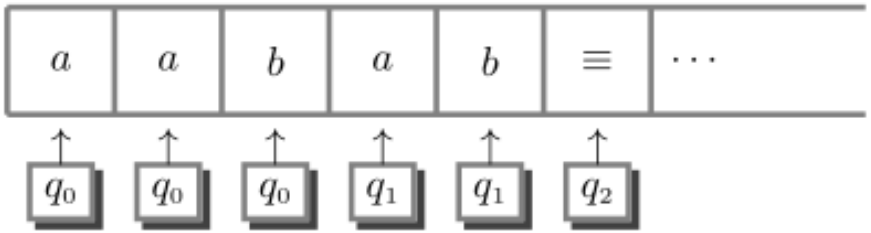
1. El computo arranca siempre en el estado inicial q_0 de la máquina.
2. El cabezal lee el primer simbolo ‘a’ y la máquina transiciona al estado q_0.
3. El cabezal lee el segundo simbolo ‘a’ y la máquina transiciona al estado q_0.
4. El cabezal lee el tercer simbolo ‘b’ y la máquina transiciona al estado q_1.
5. El cabezal lee el cuarto simbolo ‘a’ y la máquina transiciona al estado q_1.

Procesando la cadena “aabab” en un AF



1. El computo arranca siempre en el estado inicial q_0 de la máquina.
2. El cabezal lee el primer simbolo ‘a’ y la máquina transiciona al estado q_0.
3. El cabezal lee el segundo simbolo ‘a’ y la máquina transiciona al estado q_0.
4. El cabezal lee el tercer simbolo ‘b’ y la máquina transiciona al estado q_1.
5. El cabezal lee el cuarto simbolo ‘a’ y la máquina transiciona al estado q_1.
6. Por último, el cabezal lee el quinto simbolo ‘b’ y la máquina transiciona al estado q_2.

Procesando la cadena "aabab" en un AF



1. El computo arranca siempre en el estado inicial q_0 de la máquina.
2. El cabezal lee el primer simbolo 'a' y la máquina transiciona al estado q_0.
3. El cabezal lee el segundo simbolo 'a' y la máquina transiciona al estado q_0.
4. El cabezal lee el tercer simbolo 'b' y la máquina transiciona al estado q_1.
5. El cabezal lee el cuarto simbolo 'a' y la máquina transiciona al estado q_1.
6. Por último, el cabezal lee el quinto simbolo 'b' y la máquina transiciona al estado q_2.
7. Si el estado q_2 es de aceptación, la cadena será aceptada por la máquina, si no lo es, entonces será rechazada.

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*
- Q *es un conjunto (no vacio y finito) de estados*

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\delta : Q \times \Sigma \to Q$, *llamada función de transición, tal que,* $\delta(q, a) = q'$ *significa que si el autómata se encuentra en el estado "q" y el cabezal lee un símbolo "a" en la cinta de entrada, entonces el autómata transiciona al estado* q'.

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\delta : Q \times \Sigma \to Q$, *llamada función de transición, tal que,* $\delta(q, a) = q'$ *significa que si el autómata se encuentra en el estado "q" y el cabezal lee un símbolo "a" en la cinta de entrada, entonces el autómata transiciona al estado* q'.

Diremos que una cadena $\alpha \in \Sigma^*$ *es* **aceptada** *por el autómata* M *sii el estado al que arriba* M, *luego de procesar todos los símbolos de la cadena* α, *partiendo desde el estado inicial* q_0, *es un estado de aceptación.*

Autómata Finito Determinístico

Definición (AFD)

Un **Automata Finito Deterministico (AFD)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada.*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\delta : Q \times \Sigma \to Q$, *llamada función de transición, tal que,* $\delta(q, a) = q'$ *significa que si el autómata se encuentra en el estado "q" y el cabezal lee un símbolo "a" en la cinta de entrada, entonces el autómata transiciona al estado* q'.

Diremos que una cadena $\alpha \in \Sigma^*$ *es* **aceptada** *por el autómata* M *sii el estado al que arriba* M, *luego de procesar todos los símbolos de la cadena* α, *partiendo desde el estado inicial* q_0, *es un estado de aceptación.*

$$AFD^{\Sigma} = \{M : M \text{ es un AFD con alfabeto de entrada } \Sigma\}$$

Ejemplo de un AFD

Sea $M_1 = (\Sigma, Q, q_0, F, \delta)$ con $\Sigma = \{a, b\}$, $Q = \{q_0, q_1, q_2\}$, $F = \{q_0, q_2\}$ y δ definida de la siguiente manera:

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_1

$$\delta(q_0, a) = q_0 \qquad \delta(q_0, b) = q_1$$
$$\delta(q_1, a) = q_1 \qquad \delta(q_1, b) = q_2$$
$$\delta(q_2, a) = q_1 \qquad \delta(q_2, b) = q_1.$$

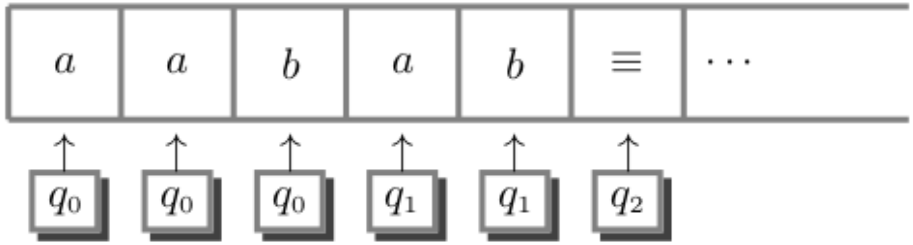
Ejemplo de un AFD

Sea $M_1 = (\Sigma, Q, q_0, F, \delta)$ con $\Sigma = \{a, b\}$, $Q = \{q_0, q_1, q_2\}$, $F = \{q_0, q_2\}$ y δ definida de la siguiente manera:

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_1

$$\delta(q_0, a) = q_0 \qquad \delta(q_0, b) = q_1$$
$$\delta(q_1, a) = q_1 \qquad \delta(q_1, b) = q_2$$
$$\delta(q_2, a) = q_1 \qquad \delta(q_2, b) = q_1.$$

Por ejemplo, la cadena $\alpha_1 =$ "*aabab*" es **aceptada** por M_1, pues el autómata arriba al estado $q_2 \in F$ luego de procesar todos sus símbolos:



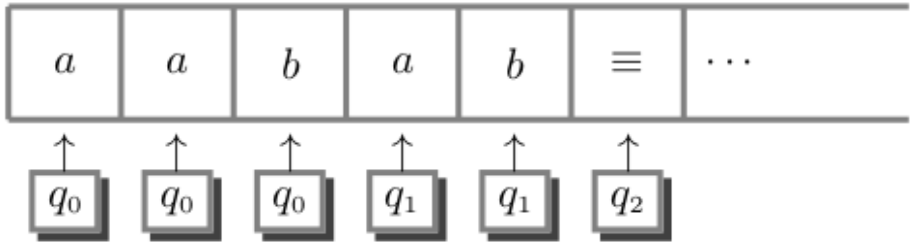
Ejemplo de un AFD

Sea $M_1 = (\Sigma, Q, q_0, F, \delta)$ con $\Sigma = \{a, b\}$, $Q = \{q_0, q_1, q_2\}$, $F = \{q_0, q_2\}$ y δ definida de la siguiente manera:

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_1

$$\delta(q_0, a) = q_0 \qquad \delta(q_0, b) = q_1$$
$$\delta(q_1, a) = q_1 \qquad \delta(q_1, b) = q_2$$
$$\delta(q_2, a) = q_1 \qquad \delta(q_2, b) = q_1.$$

Por ejemplo, la cadena $\alpha_1 =$ "*aabab*" es **aceptada** por M_1, pues el autómata arriba al estado $q_2 \in F$ luego de procesar todos sus símbolos:



Mientras que la cadena $\alpha_2 =$ "*aababa*" es **rechazada** por M_1, pues el autómata arriba al estado $q_1 \notin F$:

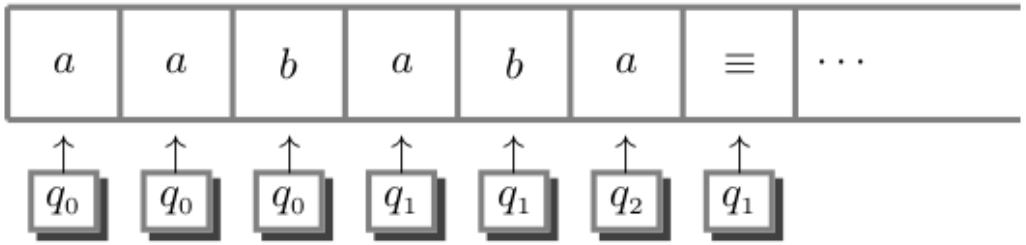


Diagrama de Transición de un AF

Representaremos un AF mediante un sistema de transición etiquetado, o sea, un grafo donde:

Diagrama de Transición de un AF

Representaremos un AF mediante un sistema de transición etiquetado, o sea, un grafo donde:

- Los nodos son los estados del AF.

Diagrama de Transición de un AF

Representaremos un AF mediante un sistema de transición etiquetado, o sea, un grafo donde:

- Los nodos son los estados del AF.
- El nodo que se corresponde con el estado inicial se marca con una flecha pequeña a su lado (por convención q_0).

Diagrama de Transición de un AF

Representaremos un AF mediante un sistema de transición etiquetado, o sea, un grafo donde:

- Los nodos son los estados del AF.
- El nodo que se corresponde con el estado inicial se marca con una flecha pequeña a su lado (por convención q_0).
- Los nodos que se corresponden con los estados de aceptación se marcarán con un doble círculo.

Diagrama de Transición de un AF

Representaremos un AF mediante un sistema de transición etiquetado, o sea, un grafo donde:

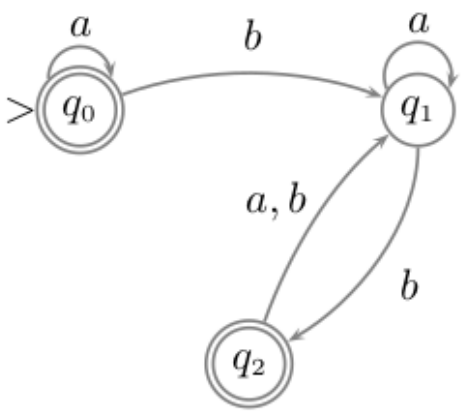
- Los nodos son los estados del AF.
- El nodo que se corresponde con el estado inicial se marca con una flecha pequeña a su lado (por convención q_0).
- Los nodos que se corresponden con los estados de aceptación se marcarán con un doble círculo.
- Existe una flecha desde el nodo q al nodo q', etiquetada con un símbolo a, siempre que $\delta(q, a) = q'$.

Diagrama de Transición de un AF

Representaremos un AF mediante un sistema de transición etiquetado, o sea, un grafo donde:

- Los nodos son los estados del AF.
- El nodo que se corresponde con el estado inicial se marca con una flecha pequeña a su lado (por convención q_0).
- Los nodos que se corresponden con los estados de aceptación se marcarán con un doble círculo.
- Existe una flecha desde el nodo q al nodo q', etiquetada con un símbolo a, siempre que $\delta(q, a) = q'$.

Por ejemplo, para el autómata anterior M_1 su diagrama de transición sería:



Lenguaje Aceptado de un AFD

Sea $M = (\Sigma, Q, q_0, F, \delta)$ un AFD, entonces definimos $\overrightarrow{\delta}(q, \alpha)$ como la función que devuelve el estado al que arriba M, luego de procesar la cadena α, partiendo del estado q:

Lenguaje Aceptado de un AFD

Sea $M = (\Sigma, Q, q_0, F, \delta)$ un AFD, entonces definimos $\overrightarrow{\delta}(q, \alpha)$ como la función que devuelve el estado al que arriba M, luego de procesar la cadena α, partiendo del estado q:

$\overrightarrow{\delta} : Q \times \Sigma^* \to Q$
$\overrightarrow{\delta}(q, \epsilon) = q$
$\overrightarrow{\delta}(q, a) = \delta(q, a)$
$\overrightarrow{\delta}(q, \alpha' a) = \delta(\overrightarrow{\delta}(q, \alpha'), a)$

Lenguaje Aceptado de un AFD

Sea $M = (\Sigma, Q, q_0, F, \delta)$ un AFD, entonces definimos $\overrightarrow{\delta}(q, \alpha)$ como la función que devuelve el estado al que arriba M, luego de procesar la cadena α, partiendo del estado q:

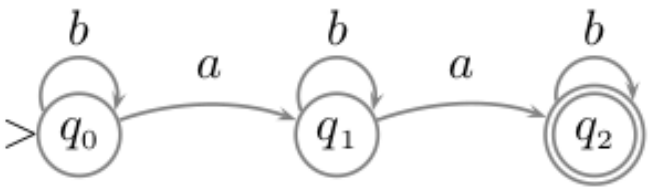
$\overrightarrow{\delta} : Q \times \Sigma^* \to Q$
$\overrightarrow{\delta}(q, \epsilon) = q$
$\overrightarrow{\delta}(q, a) = \delta(q, a)$
$\overrightarrow{\delta}(q, \alpha' a) = \delta(\overrightarrow{\delta}(q, \alpha'), a)$

Luego, definimos el **lenguaje aceptado** de M, denotado con $L(M)$, como el conjunto de todas cadenas aceptadas por M:

$$L(M) = \{\alpha \in \Sigma^* : \overrightarrow{\delta}(q_0, \alpha) \in F\}$$

Ejemplos de L(M)

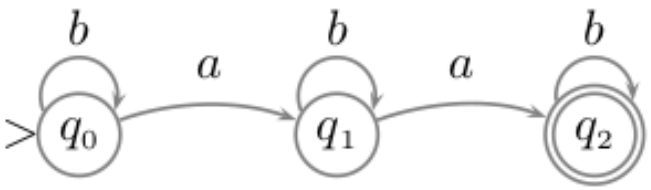
Por ejemplo, para el siguiente AFD M_1:



Tenemos que $L(M_1) = b^*ab^*ab^*$ (el lenguaje de todas las cadenas del alfabeto que tienen exactamente dos símbolos "a").

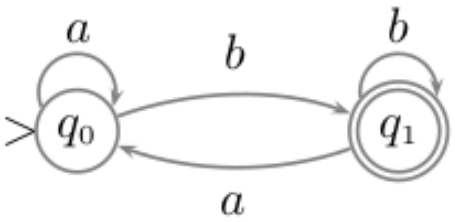
Ejemplos de L(M)

Por ejemplo, para el siguiente AFD M_1:



Tenemos que $L(M_1) = b^*ab^*ab^*$ (el lenguaje de todas las cadenas del alfabeto que tienen exactamente dos símbolos "a").

Para el AFD M_2:



Tenenemos que $L(M_2) = (a + b)^*b$ (el lenguaje de todas las cadenas del alfabeto que terminan con un símbolo "b").

Determinismo vs No-Determinismo

- **<u>Determinismo</u>**: dado un estado y un símbolo, existe un único estado al cual el autómata puede transicionar, por lo tanto, la traza de procesamiento de una cadena es única.

$$\delta(q, a) = q' \implies \overrightarrow{\delta}(q, \alpha) \text{ es un único estado}$$

Determinismo vs No-Determinismo

- **<u>Determinismo</u>**: dado un estado y un símbolo, existe un único estado al cual el autómata puede transicionar, por lo tanto, la traza de procesamiento de una cadena es única.

$$\delta(q, a) = q' \implies \overrightarrow{\delta}(q, \alpha) \text{ es un único estado}$$

´

- **<u>No-Determinismo</u>**: dado un estado y un símbolo, existen varios estados a los cuales el autómata puede transicionar, por lo tanto, existen varias trazas de procesamiento para una misma cadena.

$$\delta(q, a) = \{q1, \ldots, q_k\} \implies \overrightarrow{\delta}(q, \alpha) \text{ es un conjunto de estados.}$$

Determinismo vs No-Determinismo

- **<u>Determinismo</u>**: dado un estado y un símbolo, existe un único estado al cual el autómata puede transicionar, por lo tanto, la traza de procesamiento de una cadena es única.

$$\delta(q, a) = q' \implies \overrightarrow{\delta}(q, \alpha) \text{ es un único estado}$$

´

- **<u>No-Determinismo</u>**: dado un estado y un símbolo, existen varios estados a los cuales el autómata puede transicionar, por lo tanto, existen varias trazas de procesamiento para una misma cadena.

$$\delta(q, a) = \{q1, \ldots, q_k\} \implies \overrightarrow{\delta}(q, \alpha) \text{ es un conjunto de estados.}$$

(Para evitar confusiones, denotaremos con Δ las funciones de transición no-deterministas, mientras que con δ las deterministas)

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$*, llamado estado inicial*

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\Delta : Q \times \Sigma \to \mathcal{P}(Q)$, *llamada función de transición, tal que,* $\Delta(q, a) = \{q_1, \ldots, q_k\}$ *significa que si el autómata se encuentra en el estado "q" y el cabezal lee un símbolo "a" en la cinta de entrada, entonces el autómata puede transicionar a cualquier estado* q_i.

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\Delta : Q \times \Sigma \to \mathcal{P}(Q)$, *llamada función de transición, tal que,* $\Delta(q, a) = \{q_1, \ldots, q_k\}$ *significa que si el autómata se encuentra en el estado "q" y el cabezal lee un símbolo "a" en la cinta de entrada, entonces el autómata puede transicionar a cualquier estado* q_i.

Diremos que una cadena $\alpha \in \Sigma^*$ *es* **aceptada** *por el autómata* M, *si existe* **al menos una forma** *de procesar la cadena* α, *partiendo desde el estado inicial* q_0, *donde el estado al que arriba* M *es un estado de aceptación.*

Autómata Finito No-Determinístico

Definición (AFN)

Un **Automata Finito No-Deterministico (AFN)** *es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

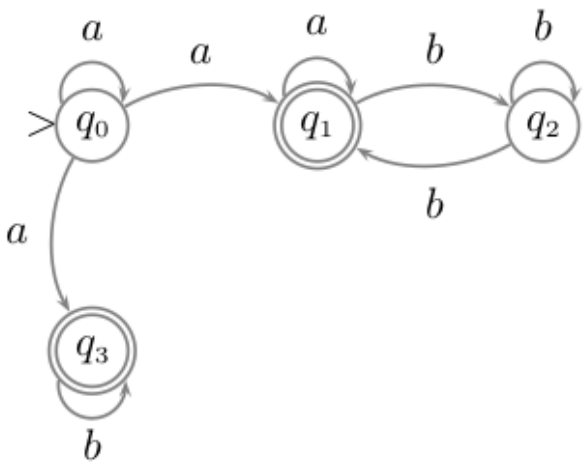
- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\Delta : Q \times \Sigma \to \mathcal{P}(Q)$, *llamada función de transición, tal que,* $\Delta(q, a) = \{q_1, \ldots, q_k\}$ *significa que si el autómata se encuentra en el estado "q" y el cabezal lee un símbolo "a" en la cinta de entrada, entonces el autómata puede transicionar a cualquier estado* q_i.

Diremos que una cadena $\alpha \in \Sigma^*$ *es* **aceptada** *por el autómata* M, *si existe* **al menos una forma** *de procesar la cadena* α, *partiendo desde el estado inicial* q_0, *donde el estado al que arriba* M *es un estado de aceptación.*

$$AFN^{\Sigma} = \{M : M \text{ es un AFN con alfabeto de entrada } \Sigma\}$$

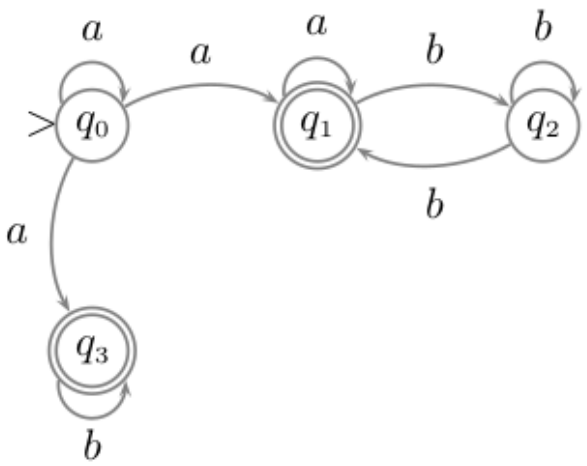
Ejemplo de AFN

Sea M_1 el siguiente AFN:

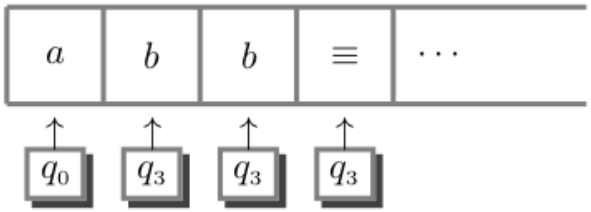


Ejemplo de AFN

Sea M_1 el siguiente AFN:

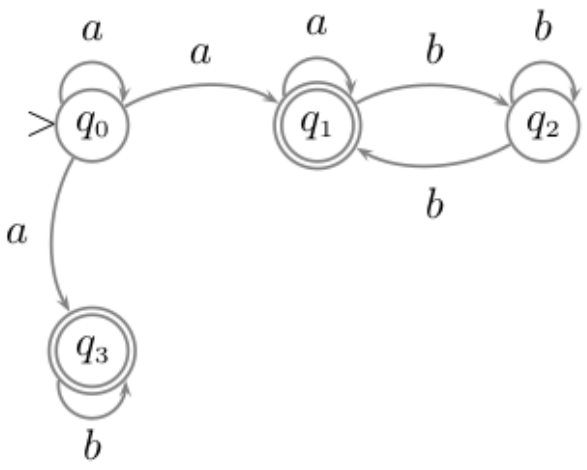


La cadena "abb" tiene una traza de procesamiento de aceptación:

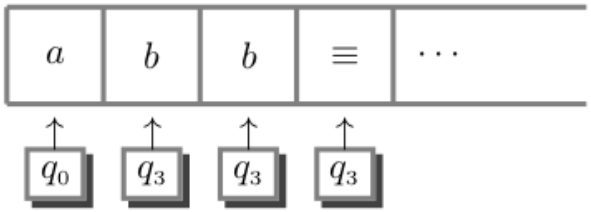


Ejemplo de AFN

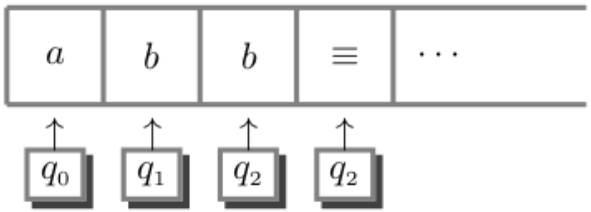
Sea M_1 el siguiente AFN:



La cadena "abb" tiene una traza de procesamiento de aceptación:



Y también, tiene una traza de procesamiento de rechazo:



Lenguaje Aceptado de un AFN

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ un AFN, primero extendemos la definición de Δ a un conjunto de estados $S \subseteq Q$:

$$\hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$$

Lenguaje Aceptado de un AFN

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ un AFN, primero extendemos la definición de Δ a un conjunto de estados $S \subseteq Q$:

$$\hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$$

Luego, definimos $\overrightarrow{\Delta}(q, a)$ como la función que devuelve el **conjunto de todos los estados posibles** al que puede arribar M luego de procesar la cadena α partiendo del estado q:

Lenguaje Aceptado de un AFN

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ un AFN, primero extendemos la definición de Δ a un conjunto de estados $S \subseteq Q$:

$$\hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$$

Luego, definimos $\overrightarrow{\Delta}(q, a)$ como la función que devuelve el **conjunto de todos los estados posibles** al que puede arribar M luego de procesar la cadena α partiendo del estado q:

$\overrightarrow{\Delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$
$\overrightarrow{\Delta}(q, \epsilon) = \{q\}$
$\overrightarrow{\Delta}(q, a) = \Delta(q, a)$
$\overrightarrow{\Delta}(q, \alpha' a) = \hat{\Delta}(\overrightarrow{\Delta}(q, \alpha'), a)$

Lenguaje Aceptado de un AFN

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ un AFN, primero extendemos la definición de Δ a un conjunto de estados $S \subseteq Q$:

$$\hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$$

Luego, definimos $\overrightarrow{\Delta}(q, a)$ como la función que devuelve el **conjunto de todos los estados posibles** al que puede arribar M luego de procesar la cadena α partiendo del estado q:

$\overrightarrow{\Delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$
$\overrightarrow{\Delta}(q, \epsilon) = \{q\}$
$\overrightarrow{\Delta}(q, a) = \Delta(q, a)$
$\overrightarrow{\Delta}(q, \alpha' a) = \hat{\Delta}(\overrightarrow{\Delta}(q, \alpha'), a)$

Para finalmente, definir el lenguaje aceptado de M como:

$$L(M) = \{\alpha \in \Sigma^* : \overrightarrow{\Delta}(q_0, \alpha) \cap F \neq \emptyset\}$$

Equivalencia entre AFD y AFN

Los AFD y los AFN son computacionalmente equivalentes.

Equivalencia entre AFD y AFN

Los AFD y los AFN son computacionalmente equivalentes.

Como toda equivalencia debemos probar una ida y una vuelta.

Equivalencia entre AFD y AFN

Los AFD y los AFN son computacionalmente equivalentes.

Como toda equivalencia debemos probar una ida y una vuelta.

Teorema ($AFD^{\Sigma} \equiv AFN^{\Sigma}$)

1. *(Ida) Si* $M \in AFD^{\Sigma}$, *entonces* $\exists M' \in AFN^{\Sigma}$ *tq* $L(M') = L(M)$.
2. *(Vuelta) Si* $M \in AFN^{\Sigma}$, *entonces* $\exists M' \in AFD^{\Sigma}$ *tq* $L(M') = L(M)$.

Equivalencia entre AFD y AFN

Los AFD y los AFN son computacionalmente equivalentes.

Como toda equivalencia debemos probar una ida y una vuelta.

Teorema ($AFD^{\Sigma} \equiv AFN^{\Sigma}$)

1. *(Ida) Si* $M \in AFD^{\Sigma}$, *entonces* $\exists M' \in AFN^{\Sigma}$ *tq* $L(M') = L(M)$.
2. *(Vuelta) Si* $M \in AFN^{\Sigma}$, *entonces* $\exists M' \in AFD^{\Sigma}$ *tq* $L(M') = L(M)$.

Demo 1. Trivial, pues un AFD puede ser visto como un AFN sin usar su no-determinismo (i.e. basta con definir $\Delta'(q, a) = \{\delta(q, a)\}$).

Equivalencia entre AFD y AFN

Los AFD y los AFN son computacionalmente equivalentes.

Como toda equivalencia debemos probar una ida y una vuelta.

Teorema ($AFD^{\Sigma} \equiv AFN^{\Sigma}$)

1. *(Ida) Si* $M \in AFD^{\Sigma}$, *entonces* $\exists M' \in AFN^{\Sigma}$ *tq* $L(M') = L(M)$.
2. *(Vuelta) Si* $M \in AFN^{\Sigma}$, *entonces* $\exists M' \in AFD^{\Sigma}$ *tq* $L(M') = L(M)$.

Demo 1. Trivial, pues un AFD puede ser visto como un AFN sin usar su no-determinismo (i.e. basta con definir $\Delta'(q, a) = \{\delta(q, a)\}$).

Demo 2. Debemos probar que **siempre se puede determinizar un AFN preservando su lenguaje**. Y el truco consiste en que los estados del AFD son conjuntos de estados del AFN, de tal manera que si el AFD se encuentra en el estado $\{q1, \ldots, q_k\}$ luego de procesar un prefijo de la cadena, entonces el AFN se encuentra en algún q_i.

Sea $M = (\Sigma, Q, q_0, F, \Delta)$, entonces tomamos $M' = (\Sigma, Q', q_0', F', \delta')$ con:

Sea $M = (\Sigma, Q, q_0, F, \Delta)$, entonces tomamos $M' = (\Sigma, Q', q_0', F', \delta')$ con:

$$Q' = \mathcal{P}(Q)$$
$$q_0' = \{q_0\}$$
$$F' = \{S \subseteq Q : S \cap F \neq \emptyset\}$$
$$\delta'(S, a) = \hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$$

Sea $M = (\Sigma, Q, q_0, F, \Delta)$, entonces tomamos $M' = (\Sigma, Q', q_0', F', \delta')$ con:

$$Q' = \mathcal{P}(Q)$$
$$q_0' = \{q_0\}$$
$$F' = \{S \subseteq Q : S \cap F \neq \emptyset\}$$
$$\delta'(S, a) = \hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$$

Claramente, vale que $\overrightarrow{\delta'}(\{q_0\}, \alpha) = \overrightarrow{\Delta}(q_0, \alpha)$ para toda α, por definición de M'.

Sea $M = (\Sigma, Q, q_0, F, \Delta)$, entonces tomamos $M' = (\Sigma, Q', q_0', F', \delta')$ con:

$Q' = \mathcal{P}(Q)$
$q_0' = \{q_0\}$
$F' = \{S \subseteq Q : S \cap F \neq \emptyset\}$
$\delta'(S, a) = \hat{\Delta}(S, a) = \bigcup_{q \in S} \Delta(q, a)$

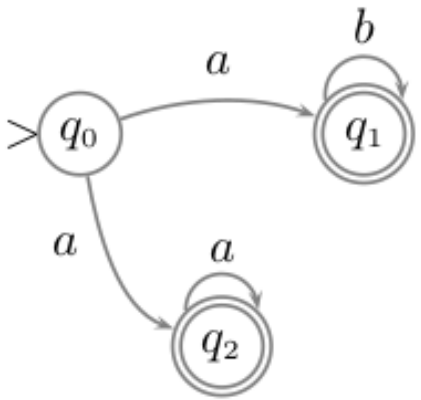
Claramente, vale que $\overrightarrow{\delta'}(\{q_0\}, \alpha) = \overrightarrow{\Delta}(q_0, \alpha)$ para toda α, por definición de M'.

Luego, $\alpha \in L(M)$
sii $\exists q \in F \in$ tal que $q \in \overrightarrow{\Delta}(q_0, \alpha)$
sii $\exists q \in F \in$ tal que $q \in \overrightarrow{\delta'}(\{q_0\}, \alpha)$
sii $\overrightarrow{\delta'}(\{q_0\}, \alpha) \in F'$
sii $\alpha \in L(M')$

$\therefore L(M) = L(M')$

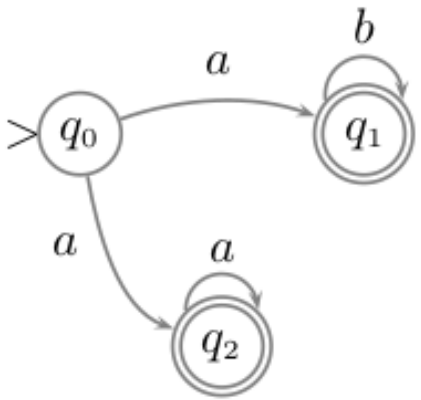
Determinizando un AFN en la práctica

Consideremos el siguiente AFN cuyo lenguaje aceptado es $a(a^* + b^*)$:



Determinizando un AFN en la práctica

Consideremos el siguiente AFN cuyo lenguaje aceptado es $a(a^* + b^*)$:

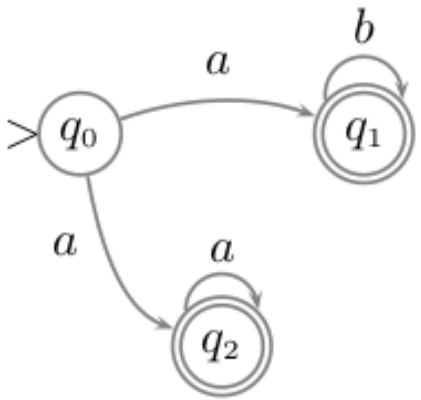


Es muy sencillo determinizar un AFN si vamos definiendo su función de transición mediante una tabla partiendo desde el estado inicial $\{q_0\}$.

	a	b
$\{q_0\}$	$\{q_1, q_2\}$	$\emptyset$

Determinizando un AFN en la práctica

Consideremos el siguiente AFN cuyo lenguaje aceptado es $a(a^* + b^*)$:

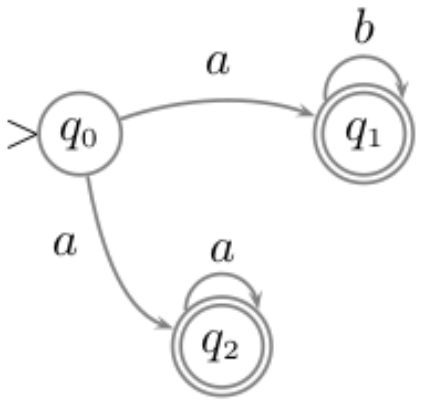


Es muy sencillo determinizar un AFN si vamos definiendo su función de transición mediante una tabla partiendo desde el estado inicial $\{q_0\}$.

	a	b
$\{q_0\}$	$\{q_1, q_2\}$	$\emptyset$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1\}$

Determinizando un AFN en la práctica

Consideremos el siguiente AFN cuyo lenguaje aceptado es $a(a^* + b^*)$:

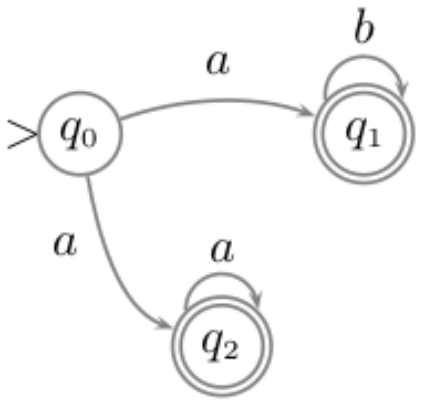


Es muy sencillo determinizar un AFN si vamos definiendo su función de transición mediante una tabla partiendo desde el estado inicial $\{q_0\}$.

	a	b
$\{q_0\}$	$\{q_1, q_2\}$	$\emptyset$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1\}$
$\{q_2\}$	$\{q_2\}$	$\emptyset$

Determinizando un AFN en la práctica

Consideremos el siguiente AFN cuyo lenguaje aceptado es $a(a^* + b^*)$:

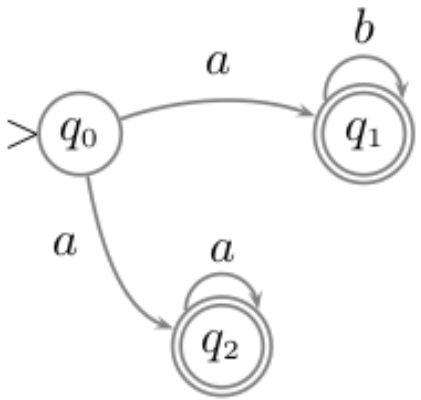


Es muy sencillo determinizar un AFN si vamos definiendo su función de transición mediante una tabla partiendo desde el estado inicial $\{q_0\}$.

	a	b
$\{q_0\}$	$\{q_1, q_2\}$	$\emptyset$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1\}$
$\{q_2\}$	$\{q_2\}$	$\emptyset$
$\{q_1\}$	$\emptyset$	$\{q_1\}$

Determinizando un AFN en la práctica

Consideremos el siguiente AFN cuyo lenguaje aceptado es $a(a^* + b^*)$:



Es muy sencillo determinizar un AFN si vamos definiendo su función de transición mediante una tabla partiendo desde el estado inicial $\{q_0\}$.

	a	b
$\{q_0\}$	$\{q_1, q_2\}$	$\emptyset$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1\}$
$\{q_2\}$	$\{q_2\}$	$\emptyset$
$\{q_1\}$	$\emptyset$	$\{q_1\}$

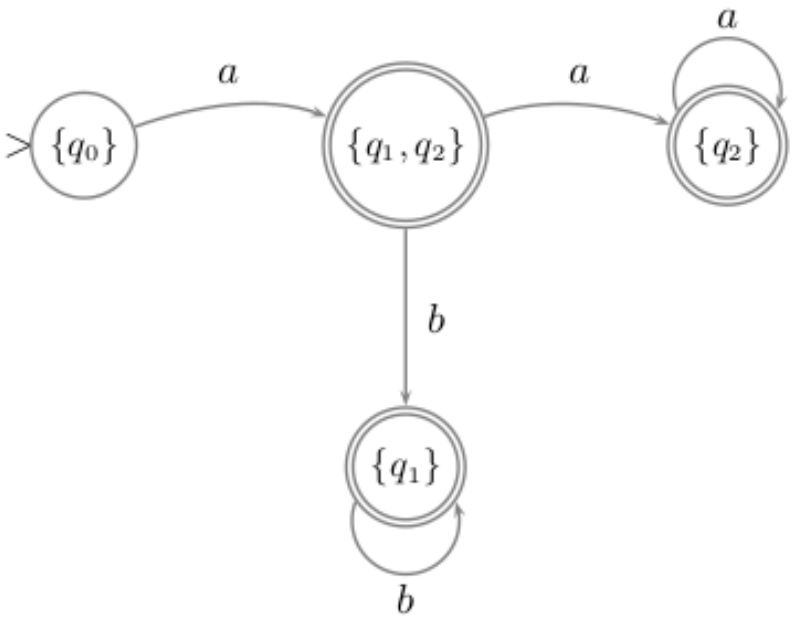
De esta manera, solo generamos los estados que son alcanzables desde el estado inicial $\{q_0\}$.

Determinizando un AFN en la práctica

Asi, tenemos el siguiente AFD:

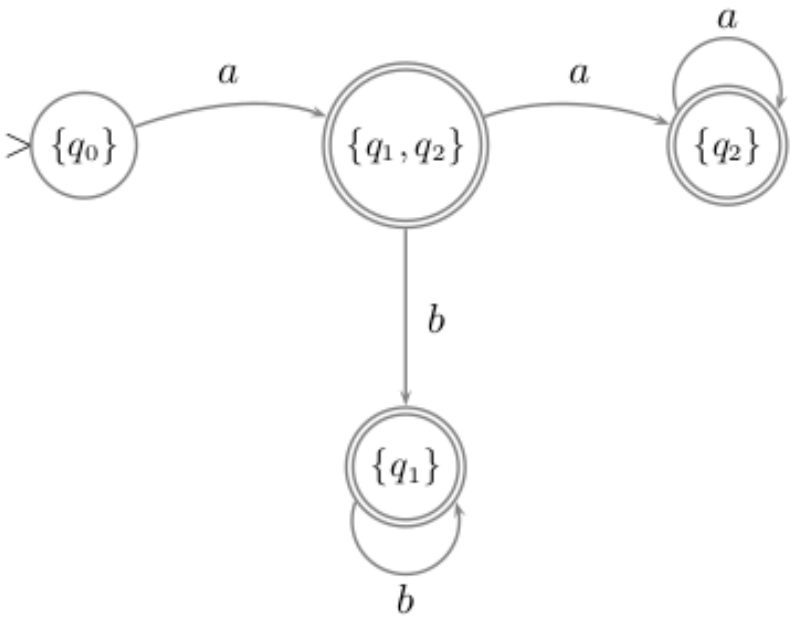
Determinizando un AFN en la práctica

Asi, tenemos el siguiente AFD:



Determinizando un AFN en la práctica

Asi, tenemos el siguiente AFD:



Y claramente, su lenguaje aceptado es $a(a^* + b^*)$, por lo tanto, es equivalente al primero.

Transiciones Espontáneas

- Permiten al autómata transicionar de estado, sin leer y desplazar el cabezal de la cinta de entrada.

Transiciones Espontáneas

- Permiten al autómata transicionar de estado, sin leer y desplazar el cabezal de la cinta de entrada.
- Una transición espontánea, si esta definida, puede tener lugar como no (por eso su nombre), en consecuencia, es en si misma una transición no-determinista.

Transiciones Espontáneas

- Permiten al autómata transicionar de estado, sin leer y desplazar el cabezal de la cinta de entrada.
- Una transición espontánea, si esta definida, puede tener lugar como no (por eso su nombre), en consecuencia, es en si misma una transición no-determinista.
- Se pueden pensar como una transición "gratis", en el sentido de que el autómata cambia de estado sin procesar o consumir un símbolo de la cinta de entrada.

Transiciones Espontáneas

- Permiten al autómata transicionar de estado, sin leer y desplazar el cabezal de la cinta de entrada.
- Una transición espontánea, si esta definida, puede tener lugar como no (por eso su nombre), en consecuencia, es en si misma una transición no-determinista.
- Se pueden pensar como una transición "gratis", en el sentido de que el autómata cambia de estado sin procesar o consumir un símbolo de la cinta de entrada.
- Las denotaremos con el símbolo ϵ, así:

$$\Delta(q, \epsilon) = \{q1, \ldots, q_k\}$$

significa que el autómata puede libremente transicionar de q a cualquier q_i sin leer símbolo alguno de la cinta de entrada (y sin desplazar su cabezal).

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla* $M = (\Sigma, Q, q_0, F, \Delta)$ *donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla $M = (\Sigma, Q, q_0, F, \Delta)$ donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla $M = (\Sigma, Q, q_0, F, \Delta)$ donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla $M = (\Sigma, Q, q_0, F, \Delta)$ donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla $M = (\Sigma, Q, q_0, F, \Delta)$ donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\Delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$, *llamada función de transición.*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla $M = (\Sigma, Q, q_0, F, \Delta)$ donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\Delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$, *llamada función de transición.*

Diremos que una cadena $\alpha \in \Sigma^*$ es **aceptada** *por el autómata M, si existe* **al menos una forma** *de procesar la cadena α, partiendo desde el estado inicial q_0, donde el estado al que arriba M es un estado de aceptación.*

Autómata Finito No-Determinístico con Transiciones Espontáneas

Definición (AFNϵ)

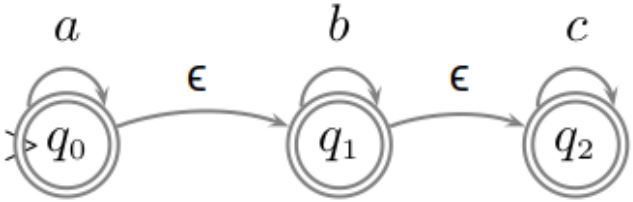
Un **Automata Finito No-Deterministico con Transiciones Espontáneas** *(AFNϵ) es un 5-upla $M = (\Sigma, Q, q_0, F, \Delta)$ donde:*

- Σ *es un alfabeto, llamado alfabeto de entrada*
- Q *es un conjunto (no vacio y finito) de estados*
- $q_0 \in Q$, *llamado estado inicial*
- $F \subseteq Q$, *llamado conjunto de estados de aceptación o finales*
- $\Delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$, *llamada función de transición.*

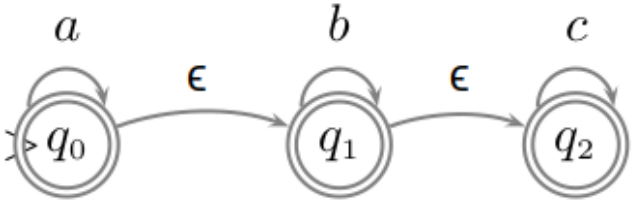
Diremos que una cadena $\alpha \in \Sigma^*$ es **aceptada** *por el autómata M, si existe* **al menos una forma** *de procesar la cadena α, partiendo desde el estado inicial q_0, donde el estado al que arriba M es un estado de aceptación.*

$$AFN\epsilon^{\Sigma} = \{M : M \text{ es un } AFN\epsilon \text{ con alfabeto de entrada } \Sigma\}$$

El AFNϵ que acepta el lenguaje $a^*b^*c^*$:

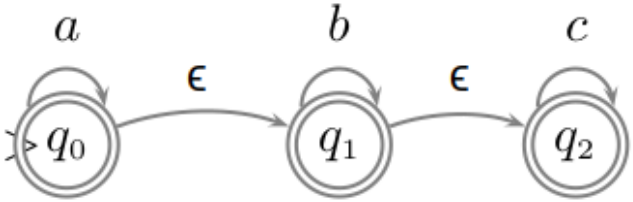


El AFNϵ que acepta el lenguaje $a^*b^*c^*$:



Las transiciones espontáneas son muy útiles en teoria de automatas, ya que, permiten muy fácilmente combinar o empalmar autómatas distintos!!!

El AFNϵ que acepta el lenguaje $a^*b^*c^*$:

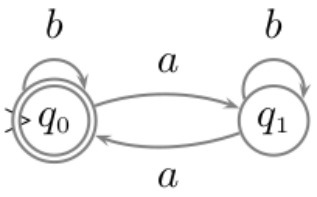


Las transiciones espontáneas son muy útiles en teoria de automatas, ya que, permiten muy fácilmente combinar o empalmar autómatas distintos!!!

En otras palabras, diseñar automatas por composicion de sub-automatas!!!

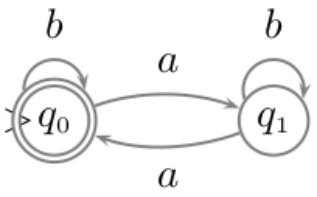
Combinando Autómatas

AFD que acepta el lenguaje de las cadenas con un número par de a's:

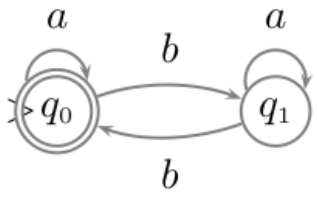


Combinando Autómatas

AFD que acepta el lenguaje de las cadenas con un número par de a's:

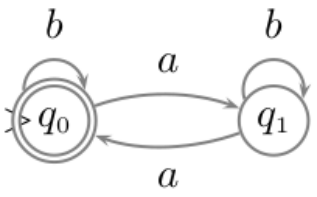


AFD que acepta el lenguaje de las cadenas con un número par de b's:

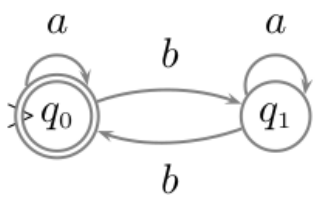


Combinando Autómatas

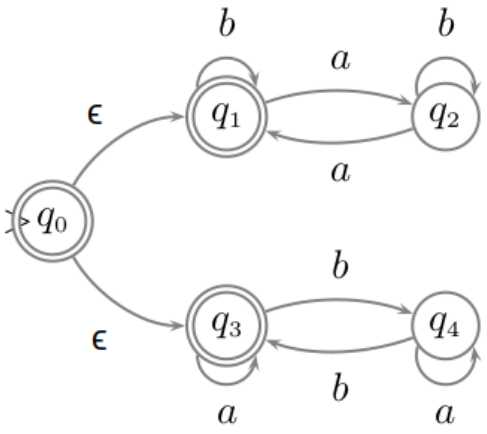
AFD que acepta el lenguaje de las cadenas con un número par de a's:



AFD que acepta el lenguaje de las cadenas con un número par de b's:



AFNϵ que acepta el lenguaje de las cadenas con un número par de a's ó b's:



Equivalencia entre AFN y AFNϵ

Los AFN y AFNϵ son computacionalmente equivalentes.

Equivalencia entre AFN y AFNϵ

Los AFN y AFNϵ son computacionalmente equivalentes.

Teorema ($AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma}$)

1. *(Ida) Si* $M \in AFN^{\Sigma}$, *entonces* $\exists M' \in AFN\epsilon^{\Sigma}$ *tq* $L(M') = L(M)$.
2. *(Vuelta) Si* $M \in AFN\epsilon^{\Sigma}$, *entonces* $\exists M' \in AFN^{\Sigma}$ *tq* $L(M') = L(M)$.

Equivalencia entre AFN y AFNϵ

Los AFN y AFNϵ son computacionalmente equivalentes.

Teorema ($AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma}$)

1. *(Ida) Si* $M \in AFN^{\Sigma}$, *entonces* $\exists M' \in AFN\epsilon^{\Sigma}$ *tq* $L(M') = L(M)$.
2. *(Vuelta) Si* $M \in AFN\epsilon^{\Sigma}$, *entonces* $\exists M' \in AFN^{\Sigma}$ *tq* $L(M') = L(M)$.

Demo 1. Trivial, pues un AFN puede ser visto como un AFNϵ sin utilizar transiciones espontáneas.

Equivalencia entre AFN y AFNϵ

Los AFN y AFNϵ son computacionalmente equivalentes.

Teorema ($AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma}$)

1. *(Ida) Si* $M \in AFN^{\Sigma}$, *entonces* $\exists M' \in AFN\epsilon^{\Sigma}$ *tq* $L(M') = L(M)$.
2. *(Vuelta) Si* $M \in AFN\epsilon^{\Sigma}$, *entonces* $\exists M' \in AFN^{\Sigma}$ *tq* $L(M') = L(M)$.

Demo 1. Trivial, pues un AFN puede ser visto como un AFNϵ sin utilizar transiciones espontáneas.

Demo 2. Debemos probar que **siempre se puede eliminar las transiciones espontáneas sin afectar el lenguaje**. El truco consiste en introducir transiciones que simulen los mismos efectos de las espontáneas. Para esto, debemos introducir primero la noción de **clausura de un estado**.

Clausura de un Estado

La **clausura** de un estado q, denotado con $[q]$, es el conjunto de estados al que puedo arribar desde q, utilizando solamente transiciones espontáneas:

Clausura de un Estado

La **clausura** de un estado q, denotado con $[q]$, es el conjunto de estados al que puedo arribar desde q, utilizando solamente transiciones espontáneas:

$$q \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q_k \Longrightarrow [q] = \{q, q_1, \ldots, q_k\}$$

Clausura de un Estado

La **clausura** de un estado q, denotado con $[q]$, es el conjunto de estados al que puedo arribar desde q, utilizando solamente transiciones espontáneas:

$$q \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q_k \Longrightarrow [q] = \{q, q_1, \ldots, q_k\}$$

Definimos formalmente la clausura de un estado q:

$$[q] = \{q' \in Q : q' \in \overrightarrow{\Delta}(q, \epsilon)\}$$

Clausura de un Estado

La **clausura** de un estado q, denotado con $[q]$, es el conjunto de estados al que puedo arribar desde q, utilizando solamente transiciones espontáneas:

$$q \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q_k \Longrightarrow [q] = \{q, q_1, \ldots, q_k\}$$

Definimos formalmente la clausura de un estado q:

$$[q] = \{q' \in Q : q' \in \overrightarrow{\Delta}(q, \epsilon)\}$$

Extendemos la definición de clausura para conjunto de estados $S \subseteq Q$:

$$[S] = \bigcup_{q \in S} [q]$$

Clausura de un Estado

La **clausura** de un estado q, denotado con $[q]$, es el conjunto de estados al que puedo arribar desde q, utilizando solamente transiciones espontáneas:

$$q \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2 \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q_k \Longrightarrow [q] = \{q, q_1, \ldots, q_k\}$$

Definimos formalmente la clausura de un estado q:

$$[q] = \{q' \in Q : q' \in \overrightarrow{\Delta}(q, \epsilon)\}$$

Extendemos la definición de clausura para conjunto de estados $S \subseteq Q$:

$$[S] = \bigcup_{q \in S} [q]$$

Para todo $q \in Q$, se cumple que:

1. $q \in [q]$
2. $[[q]] = [q]$

Ahora si probemos la vuelta (2).

Ahora si probemos la vuelta (2).

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ el AFNϵ, entonces podemos tomar el AFN $M' = (\Sigma, Q', q_0', F', \Delta')$ donde:

Ahora si probemos la vuelta (2).

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ el AFNϵ, entonces podemos tomar el AFN $M' = (\Sigma, Q', q_0', F', \Delta')$ donde:

Se preservan todos los estados y el estado inicial del AFNϵ:

$Q' = Q$
$q_0' = q_0$

Ahora si probemos la vuelta (2).

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ el AFNϵ, entonces podemos tomar el AFN $M' = (\Sigma, Q', q_0', F', \Delta')$ donde:

Se preservan todos los estados y el estado inicial del AFNϵ:

$Q' = Q$
$q_0' = q_0$

Los estados finales son aquellos tal que su clausura contiene al menos un estado de aceptación del AFNϵ:

$F' = \{q \in Q : [q] \cap F \neq \emptyset\}$

Ahora si probemos la vuelta (2).

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ el AFNϵ, entonces podemos tomar el AFN $M' = (\Sigma, Q', q_0', F', \Delta')$ donde:

Se preservan todos los estados y el estado inicial del AFNϵ:

$Q' = Q$
$q_0' = q_0$

Los estados finales son aquellos tal que su clausura contiene al menos un estado de aceptación del AFNϵ:

$F' = \{q \in Q : [q] \cap F \neq \emptyset\}$

Y por último, intuitivamente si en el AFNϵ tenemos un camino $q \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q_1 \xrightarrow{a} q_2 \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q'$, entonces en el AFN definimos la transición $q \xrightarrow{a} q'$:

$\Delta'(q, a) = [\hat{\Delta}([q], a)]$

Ahora si probemos la vuelta (2).

Sea $M = (\Sigma, Q, q_0, F, \Delta)$ el AFNϵ, entonces podemos tomar el AFN $M' = (\Sigma, Q', q_0', F', \Delta')$ donde:

Se preservan todos los estados y el estado inicial del AFNϵ:

$Q' = Q$
$q_0' = q_0$

Los estados finales son aquellos tal que su clausura contiene al menos un estado de aceptación del AFNϵ:

$F' = \{q \in Q : [q] \cap F \neq \emptyset\}$

Y por último, intuitivamente si en el AFNϵ tenemos un camino $q \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q_1 \xrightarrow{a} q_2 \xrightarrow{\epsilon} \ldots \xrightarrow{\epsilon} q'$, entonces en el AFN definimos la transición $q \xrightarrow{a} q'$:

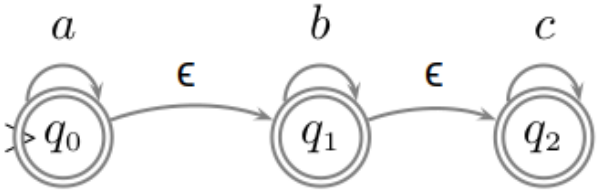
$\Delta'(q, a) = [\hat{\Delta}([q], a)]$

Luego, $L(M) = L(M')$.

Eliminando Transiciones Espontaneas en la práctica

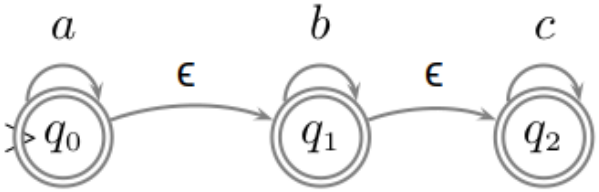
Eliminando Transiciones Espontaneas en la práctica

Por ejemplo, sea M el siguiente AFNϵ con L(M) $= a^* b^* c^*$.



Eliminando Transiciones Espontaneas en la práctica

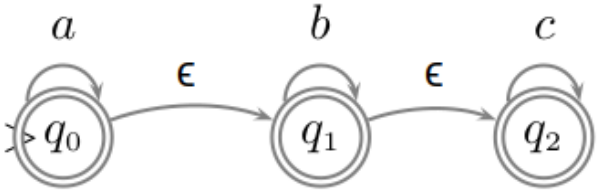
Por ejemplo, sea M el siguiente AFNϵ con L(M) $= a^* b^* c^*$.



Obtengamos el AFN equivalente eliminando sus transiciones espontáneas.

Eliminando Transiciones Espontaneas en la práctica

Por ejemplo, sea M el siguiente AFNϵ con L(M) $= a^* b^* c^*$.

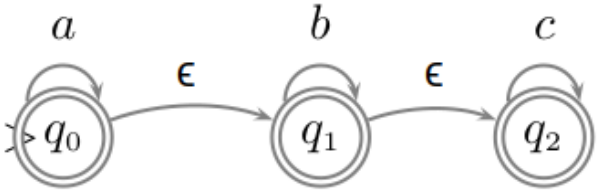


Obtengamos el AFN equivalente eliminando sus transiciones espontáneas.

Calculamos las clausuras de los estados:
$[q_0] = \{q_0, q_1, q_2\}$
$[q_1] = \{q_1, q_2\}$
$[q_2] = \{q_2\}$

Eliminando Transiciones Espontaneas en la práctica

Por ejemplo, sea M el siguiente AFNϵ con L(M) $= a^* b^* c^*$.



Obtengamos el AFN equivalente eliminando sus transiciones espontáneas.

Calculamos las clausuras de los estados:
$[q_0] = \{q_0, q_1, q_2\}$
$[q_1] = \{q_1, q_2\}$
$[q_2] = \{q_2\}$

Calculamos los estados de aceptación:
$F' = \{q_0, q_1, q_2\}$

Y por último, calculamos la función de transición mediante una tabla:

Y por último, calculamos la función de transición mediante una tabla:

	a	b	c
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$

Y por último, calculamos la función de transición mediante una tabla:

	a	b	c
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$

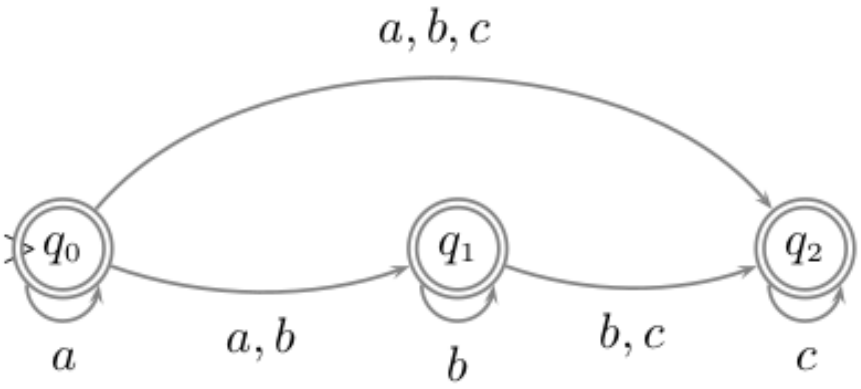
Y por último, calculamos la función de transición mediante una tabla:

	a	b	c
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$

Y por último, calculamos la función de transición mediante una tabla:

	a	b	c
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$

Obteniendo finalmente el siguiente AFN M' equivalente pues $L(M') = a^* b^* c^*$.



Resumen

- Presentamos tres tipos de autómatas finitos (AFD, AFN, AFNϵ) y probamos que son todos computacionalmente equivalentes :

$$AFD^{\Sigma} \equiv AFN^{\Sigma} \text{ y } AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma} \therefore AFD^{\Sigma} \equiv AFN\epsilon^{\Sigma}$$

Resumen

- Presentamos tres tipos de autómatas finitos (AFD, AFN, AFNϵ) y probamos que son todos computacionalmente equivalentes :
$$AFD^{\Sigma} \equiv AFN^{\Sigma} \text{ y } AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma} \therefore AFD^{\Sigma} \equiv AFN\epsilon^{\Sigma}$$
- En la práctica, convertimos un AFNϵ a su AFD equivalente, aplicando primero la técnica de eliminación de transiciones espontáneas, y luego, determinización.

Resumen

- Presentamos tres tipos de autómatas finitos (AFD, AFN, AFNϵ) y probamos que son todos computacionalmente equivalentes :
$$AFD^{\Sigma} \equiv AFN^{\Sigma} \text{ y } AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma} \therefore AFD^{\Sigma} \equiv AFN\epsilon^{\Sigma}$$
- En la práctica, convertimos un AFNϵ a su AFD equivalente, aplicando primero la técnica de eliminación de transiciones espontáneas, y luego, determinización.
- Por ser equivalentes, hablaremos directamente de autómata finito (AF) en general, sin especificar su tipo:
$$AF^{\Sigma} = AFD^{\Sigma} \cup AFN^{\Sigma} \cup AFN\epsilon^{\Sigma}$$

Resumen

- Presentamos tres tipos de autómatas finitos (AFD, AFN, AFNϵ) y probamos que son todos computacionalmente equivalentes :
$$AFD^{\Sigma} \equiv AFN^{\Sigma} \text{ y } AFN^{\Sigma} \equiv AFN\epsilon^{\Sigma} \therefore AFD^{\Sigma} \equiv AFN\epsilon^{\Sigma}$$
- En la práctica, convertimos un AFNϵ a su AFD equivalente, aplicando primero la técnica de eliminación de transiciones espontáneas, y luego, determinización.
- Por ser equivalentes, hablaremos directamente de autómata finito (AF) en general, sin especificar su tipo:
$$AF^{\Sigma} = AFD^{\Sigma} \cup AFN^{\Sigma} \cup AFN\epsilon^{\Sigma}$$
- Ahora estamos en condiciones de probar (próxima clase) que los AF caracterizan a los lenguajes regulares:
$$LR^{\Sigma} \equiv AF^{\Sigma}$$

Bibliografía

Rodrigo De Castro Korgi.
"Teoria de la Computación". Lenguajes, Autómatas, Gramáticas.
Capítulo 2: Autómatas Finitos. Sección 2.1 hasta 2.7.